Teacher:	Ray Joshua Balingkit	**Subject:**	Computer Science
Week:	13 (Term 1)	**Year:**	10-11
Link:		**Cover:**	

LESSON OVERVIEW

What should students know/do by the end of this week's lessons?

Students will develop a systematic approach to testing and evaluating programs against stated requirements. They will record expected and actual results and use this evidence to identify and correct a logic error. Students will select normal, boundary and invalid test data, justify how each case checks a requirement and distinguish a successful test from a claim that the whole program is correct.

End-of-Week Assessment/Check: How will you know they got it?

Teacher questioning, discussion of worked examples, the completed student tasks and the final checks in both lessons. Check evidence before and after the correction and one justified evaluation statement. Require expected and actual outcomes and explain how a test exposes an incorrect comparison. Check that students can explain their reasoning and apply the skill without relying on the worked example.

LESSON SLOTS - MONDAY & FRIDAY

Lesson	Date	Learning Objective	The Lesson Flow	Differentiation
1	16 November 2026	By the end of this lesson, students will be able to distinguish testing from evaluation and record useful test evidence.	**H:** Show a working program that still does not meet its stated purpose. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain expected and actual results, error diagnosis and evaluation against requirements. Distinguish showing that a program runs from showing that it meets its requirements. Explain how a repeatable test records its purpose, input, expected result, actual result and whether the requirement is satisfied. **WD:** Test a score classifier and link each case to a stated requirement. Use a classifier that awards a pass only above 50 even though the requirement says 50 or above. Predict	**Support:** Provide the requirements and a test-table template with one example completed. Highlight the difference between expected and actual results. Guide students to describe the observed error before changing the code. **Extension:** Students explain why rerunning only the previously failing case is insufficient. They choose an additional case that checks whether the correction has introduced another error.

			the outcome for 50, run the test and correct the comparison. Rerun an ordinary case to check the change. **YD:** Create a test table, run the supplied program and correct a discovered logic error. Students produce a completed test table and keep the faulty and corrected results. Their evaluation refers to a stated requirement, cites the evidence and identifies one remaining limitation or improvement. **C:** Check evidence before and after the correction and one justified evaluation statement. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
2	20 November 2026	By the end of this lesson, students will be able to select normal boundary and invalid test data for a program.	**H:** Ask why testing only a score of 50 misses important errors. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain normal values, valid limits and invalid values outside the accepted range. Explain that normal data lies within the expected range, boundary data checks the permitted limits and invalid data violates the rule. Some conventions also group values immediately outside a limit with boundary testing; always state the rule and expected outcome. **WD:** Select 0, 100, -1 and 101 for a permitted range of 0–100. For an inclusive 0–100 range, choose 50 as normal, 0 and 100 as	**Support:** Provide a labelled number line and prepared test values to sort. Model one expected result and give sentence stems linking a value to the requirement. Support the recording process without choosing every test for students. **Extension:** Students add a decimal case and explain whether it is valid under an integer-only requirement. They distinguish testing a validation rule from proving that an entered score is factually correct.



			valid boundaries, and -1 and 101 as invalid numeric values. Identify text as another invalid category requiring a different handling strategy. **YD:** Test a range-checking program using normal, boundary and invalid numeric data; identify text input as a further limitation. Students justify each selected test, run the checker and record the output. They modify a faulty comparison and show which test detects the error; document text input as an unresolved limitation if conversion handling is not implemented. **C:** Require expected and actual outcomes and explain how a test exposes an incorrect comparison. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
3				Support: Extension:
4				Support: Extension:

MATERIALS NEEDED FOR THE WEEK

- Pearson Edexcel International GCSE Computer Science Student Book: Chapter 11
- Teacher computer and classroom display
- Classroom computers with Python 3 editor and individual save folders
- Prepared algorithm, tracing or Python practice tasks aligned with the lesson objectives
- Whiteboard and markers
- Student notebooks and pencils
- Success criteria, test tables and teacher observation checklist



END-OF-WEEK REFLECTION

- What worked well?

- What flopped or confused students?

- What needs to be re-taught or adjusted next week?